

Student Dormitory Management System

Technical Analysis and Design Report

Project 2

Version 1.0
June 2026

Abstract

This report defines the analysis and design for a web-based Student Dormitory Management System. The proposed system centralizes student records, room and bed inventory, check-in and check-out workflows, billing, payment review, announcements, notifications, and operational reporting. The design is based on the provided project proposal and additional project notes, with a focus on producing an implementable MVP before adding future enhancements.

Contents

1	Intro	4
1.1	Purpose	4
1.2	Problem Summary	4
1.3	Proposed Solution	4
1.4	Document Scope	4
2	Stakeholder and Context Analysis	4
2.1	Stakeholders	4
2.2	Operational Context	5
2.3	Context Diagram	5
2.4	Assumptions and Constraints	6
3	Requirement Analysis	6
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	7
3.3	Requirement Prioritization	8
4	Analysis Models	8
4.1	Use Case Diagram	8
4.2	Activity Diagrams	8
4.3	Sequence Diagrams	11
4.4	State Diagrams	13
4.5	Domain Model	14
4.6	Data Flow Diagram	15
5	System Design Overview	16
5.1	Design Approach	16
5.2	MVP Subsystems	16
5.3	Planned Django App Layout	17
5.4	High-Level Request Flow	17
6	Architectural Design	18
6.1	Architecture Style	18
6.2	Design Goals and Principles	18
6.3	Design Patterns Applied	19
6.4	Architecture Diagram	20
6.5	Package and Layer Decomposition	20
6.6	Component Diagram	21
6.7	Technology Stack	21
6.8	Deployment Overview	22
7	Data / Class Design	23
7.1	Class Diagram	23
7.2	Database Schema	24
7.3	Database Constraints and Indexes	25
7.4	Entity Relationships	26
7.5	Data Dictionary	28
7.6	Important Data Structures	29

8	Interface Design	29
8.1	User Interface Overview	29
8.2	Information Architecture and Sitemap	29
8.3	Core User Flow	30
8.4	Front-end Component Structure	30
8.5	Screenshot Replacement Plan	31
8.6	Dashboard Placeholder	31
8.7	Payment Review Placeholder	32
8.8	API Design	32
8.9	Representative API Payloads	33
8.10	Input and Output Formats	34
8.11	External Interfaces	34
9	Component-Level / Detailed Design	35
9.1	Service Method Map	35
9.2	Identity and Access Module	35
9.3	Resident Management Module	36
9.4	Room Inventory Module	36
9.5	Assignment Module	36
9.6	Check-In / Check-Out Module	37
9.7	Billing Module	37
9.8	Payment Review Module	38
9.9	Notification Module	38
9.10	Reporting Module	39
9.11	Demo Data Module	39
10	Quality Attributes and Design Rationale	39
11	Requirement–Design Traceability	40
12	Validation / Testing Strategy	41
12.1	Design Review	41
12.2	Test Plan Overview	41
12.3	Key Test Cases	42
12.4	Acceptance Criteria	42
13	Conclusion	43

1 Intro

1.1 Purpose

The purpose of this report is to transform the initial Student Dormitory Management System proposal into an implementable software design. The report will be used as the technical baseline for the later demo implementation. It describes the business context, requirements, analysis models, architecture, data design, interfaces, detailed modules, design rationale, traceability, and validation strategy.

1.2 Problem Summary

Dormitory operations commonly rely on spreadsheets, paper forms, messaging applications, and separate financial records. This makes it difficult to maintain accurate room occupancy, track student residence history, reconcile payments, follow up overdue invoices, and communicate operational updates. The proposed system reduces this fragmentation by providing a secure, role-based web platform for dormitory operations.

1.3 Proposed Solution

The proposed solution is a modular web application with the following MVP capabilities:

- Centralized student and resident profile management.
- Building, floor, room, and bed inventory management.
- Automatic room and bed assignment based on capacity, gender policy, room status, and student preferences.
- Check-in and check-out workflows with inspection records, penalties, and occupancy history.
- Billing, invoice generation, payment evidence upload, payment evidence aggregation, email notification, and admin confirmation.
- Announcements and notifications for students and staff.
- Dashboard and reports for occupancy, outstanding balances, and operational status.
- Synthetic data generation for demo and testing.

1.4 Document Scope

This report focuses on the design of the MVP and keeps future enhancements clearly separated. Future enhancements include online housing applications, roommate matching, visitor management, payment gateway integration, mobile access, and university system integration.

2 Stakeholder and Context Analysis

2.1 Stakeholders

Stakeholder	Main Goals	System Concerns
Student	View room assignment, invoices, announcements, and payment status. Upload payment evidence and request check-out.	Ease of use, accurate balance, clear deadlines, privacy of personal and financial data.
Dorm Admin	Manage students, rooms, beds, assignments, check-in, check-out, inspections, and announcements.	Correct occupancy, no double booking, fast assignment, traceable decisions, operational dashboards.
Finance Staff	Generate invoices, review payments, track deposits, penalties, and outstanding balances.	Accurate receivables, audit trail, reconciliation, overdue account visibility.
System Administrator	Configure users, roles, master data, backups, and deployment.	Security, reliability, maintainability, access control, recovery.
University Management	Review aggregate occupancy and financial reports.	Management visibility, policy compliance, reliable reporting.

2.2 Operational Context

The system operates inside a university dormitory environment. Students interact with the system through a web portal. Dorm administrators manage housing operations. Finance staff manage invoices, deposits, fees, and payment confirmation. Email is used for reminders and payment review summaries. The database is the authoritative source for room inventory, resident status, invoices, payments, and audit history.

2.3 Context Diagram

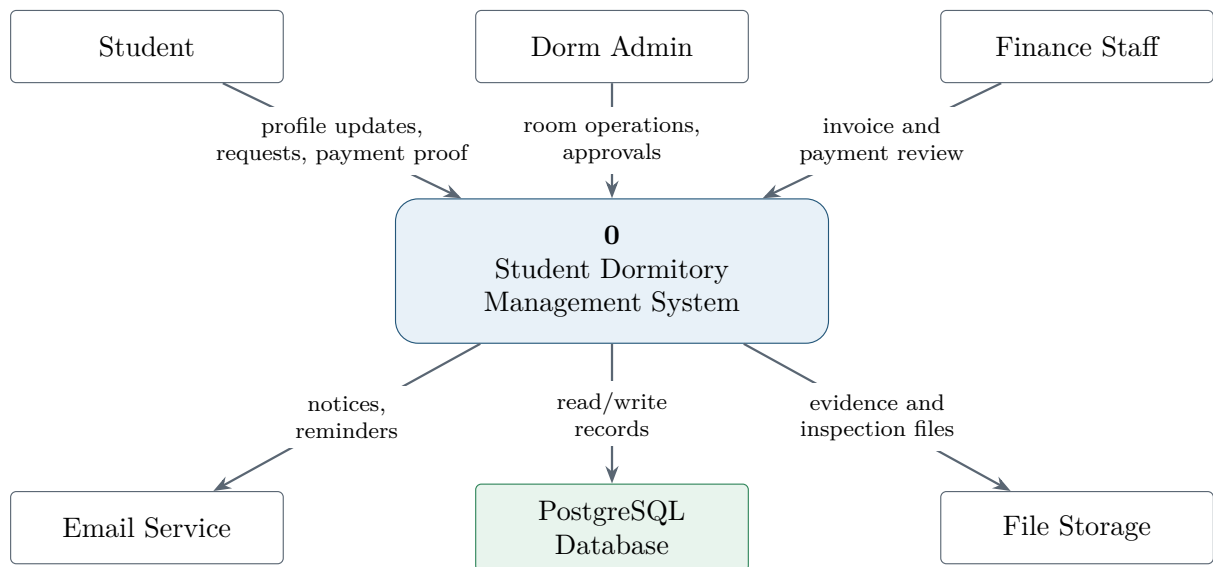


Figure 1: System context diagram

2.4 Assumptions and Constraints

- The MVP is a web application accessed by desktop and mobile browsers.
- The first implementation will use Django templates, Bootstrap, HTMX, PostgreSQL, and email integration.
- Payment gateway integration is outside the MVP. Students upload evidence, and an authorized staff member confirms payment status.
- Synthetic data is acceptable for demo accounts, rooms, invoices, and payments.
- Role-based authorization is mandatory for all protected pages and APIs.

3 Requirement Analysis

3.1 Functional Requirements

ID	Name	Description	Priority
FR-01	Authentication and roles	Users can sign in and access features according to role: Student, Dorm Admin, Finance Staff, and System Administrator.	Must
FR-02	Student profile management	Staff can create, import, update, search, and deactivate student records. Students can view their own profile.	Must
FR-03	Room and bed inventory	Admins can manage buildings, floors, rooms, beds, status, capacity, and occupancy.	Must
FR-04	Automatic assignment	The system recommends or assigns an available bed using policy rules and assignment scoring.	Must
FR-05	Manual assignment override	Admins can manually assign or reassign beds with reason logging.	Should
FR-06	Check-in workflow	Staff can finalize move-in, record handover information, activate the occupancy record, and generate initial charges.	Must
FR-07	Check-out workflow	Staff can record departure request, inspection, penalties, deposit adjustment, and return the bed to inventory.	Must
FR-08	Billing and invoices	Finance staff can generate rent, deposit, fee, and penalty invoices for students.	Must
FR-09	Payment evidence upload	Students can upload payment proof for invoices. The system stores files and links them to payment records.	Must
FR-10	Payment review and confirmation	The system aggregates submitted payment evidence, sends review notification by email, and only authorized staff can confirm paid or rejected status.	Must

ID	Name	Description	Priority
FR-11	Notifications and announcements	Admins can broadcast announcements and send reminders for invoices, check-in, check-out, and overdue balances.	Should
FR-12	Dashboard and reports	Staff can view occupancy, available beds, active residents, invoices, payments, and overdue accounts.	Must
FR-13	Audit trail	Critical actions such as assignment, payment confirmation, invoice updates, and check-out settlement are logged.	Should
FR-14	Synthetic demo data	The system can seed realistic demo data for students, rooms, invoices, payments, and occupancy history.	Must

3.2 Non-Functional Requirements

ID	Quality Attribute	Requirement
NFR-01	Security	All protected operations require authentication and role-based authorization. Passwords are hashed. Sensitive records are visible only to permitted roles.
NFR-02	Reliability	Assignment and payment confirmation use database transactions to prevent double booking and inconsistent payment status.
NFR-03	Usability	Common workflows should be completed in a small number of steps, with searchable tables and clear status labels.
NFR-04	Performance	Core list pages should respond within 2 seconds for demo-scale data and remain indexed for larger data.
NFR-05	Maintainability	The codebase should be organized by Django apps and service classes, with business rules separated from views.
NFR-06	Extensibility	Future modules such as visitor management, payment gateways, and university SSO should be addable without rewriting the core model.
NFR-07	Auditability	Financial and occupancy state changes should preserve actor, timestamp, old status, new status, and reason.
NFR-08	Portability	The demo should run locally using environment variables and should support containerized deployment.

3.3 Requirement Prioritization

The MVP includes the modules needed to run dormitory operations end to end: users, students, rooms, assignment, check-in, billing, payment review, check-out, announcements, dashboard, and demo data. Future enhancements are intentionally kept outside the MVP to reduce implementation risk.

4 Analysis Models

4.1 Use Case Diagram

The use case model defines the externally visible behavior of the MVP. The system boundary separates user goals from implementation modules. Students mainly consume information and submit evidence or requests; dorm admins operate residence workflows; finance staff control invoices and payment decisions. This separation is important because it drives role-based authorization in the later design.

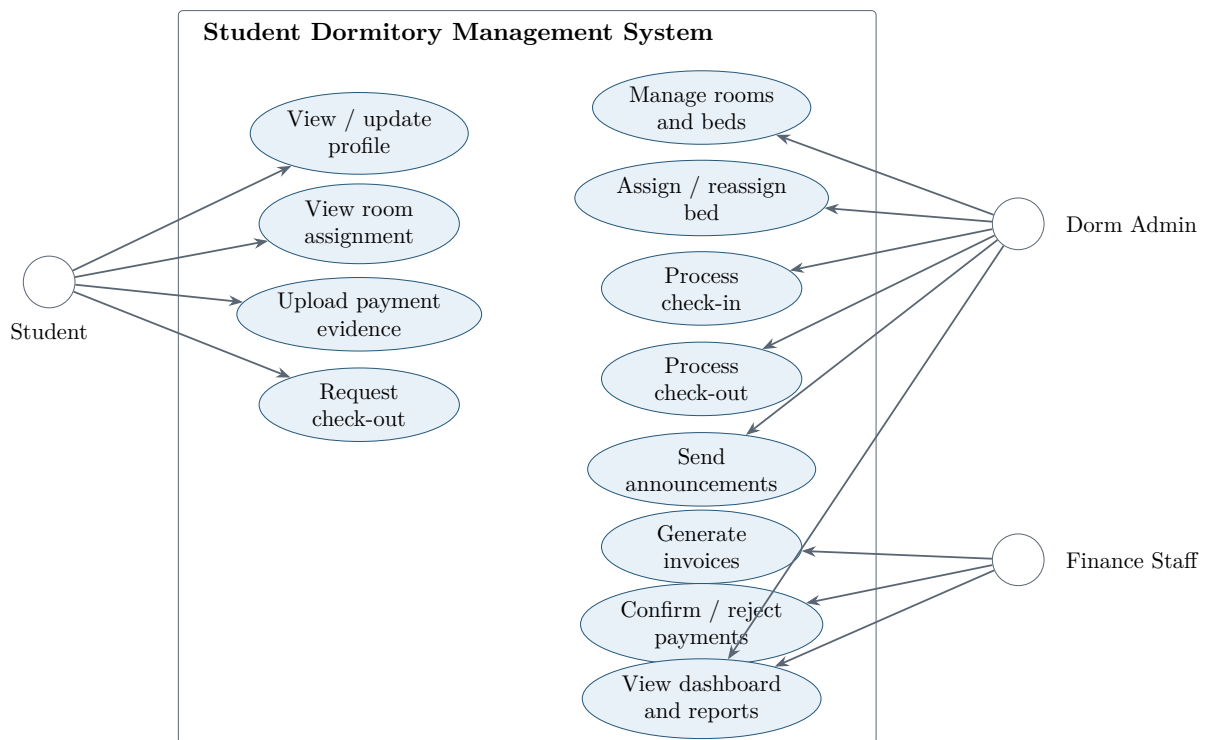


Figure 2: Use case diagram for the MVP

The use case diagram also identifies authorization boundaries. For example, a student can upload evidence but cannot confirm a payment. Finance can confirm payments but should not directly change room inventory. Those boundaries later become permission decorators, navigation rules, and negative tests.

4.2 Activity Diagrams

Activity diagrams describe workflow ordering and decision points. They are used here for the three workflows most likely to create inconsistent data if implemented incorrectly: onboarding, payment review, and check-out settlement.

Student Onboarding and Assignment

The onboarding activity starts from a student record and ends with an active occupancy record. The key decision points are bed availability and payment confirmation. If no bed is available, the student enters a waiting-list path. If payment is not confirmed, the system sends reminders and does not activate the bed.

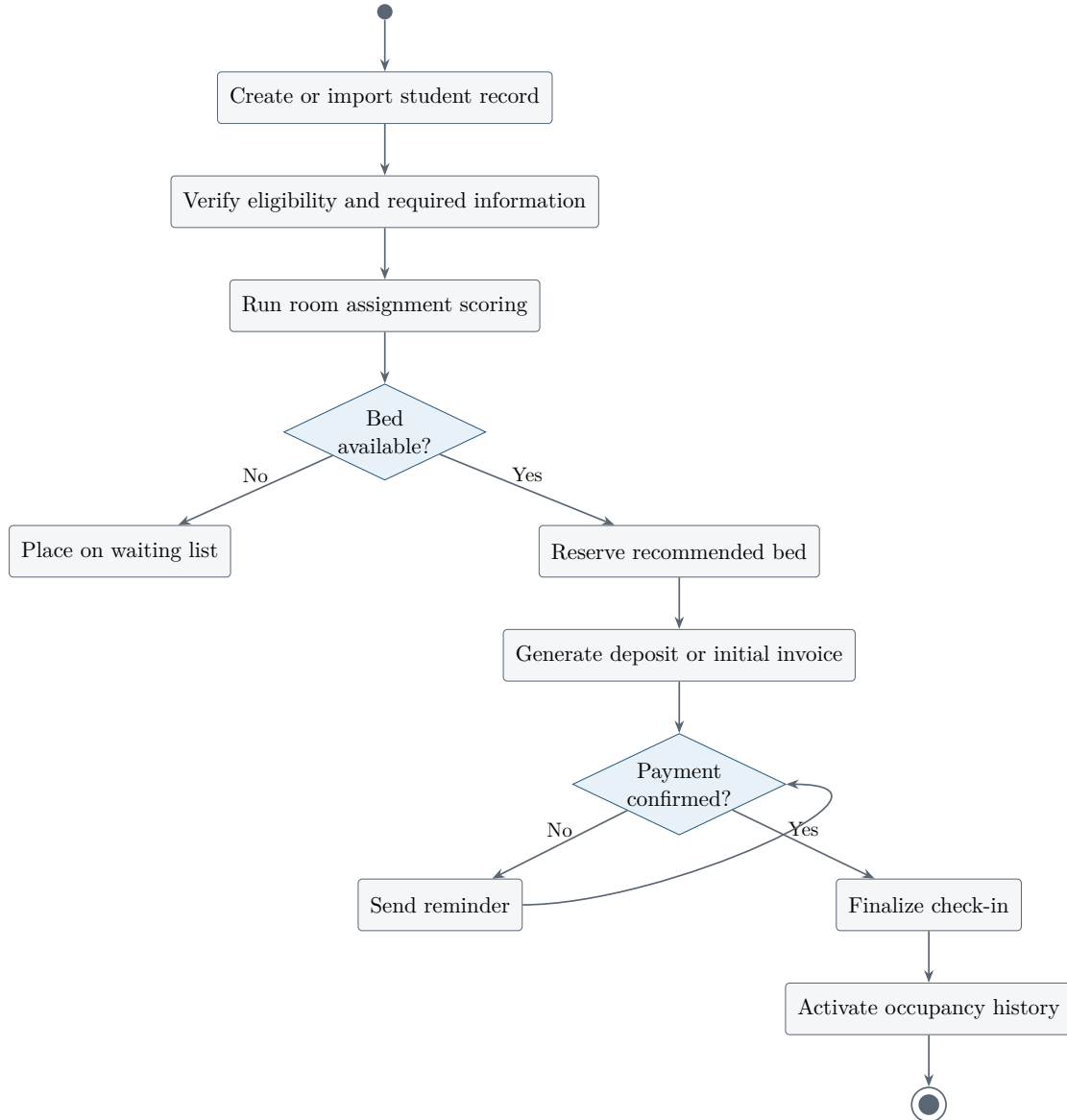


Figure 3: Activity diagram for onboarding

Payment Evidence Review

The payment workflow is intentionally two-step: students submit evidence, while finance staff make the financial decision. Pending evidence does not reduce outstanding balance. This protects the receivable record from accidental or fraudulent student-side updates.

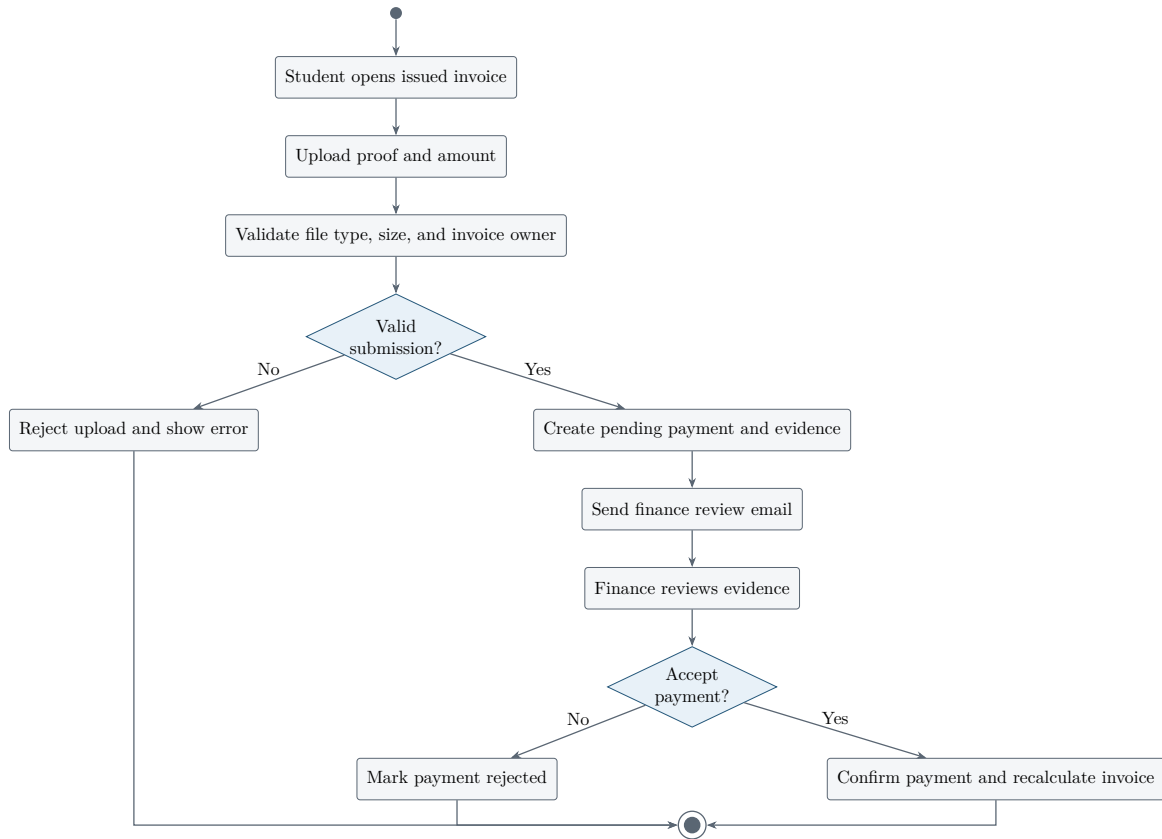


Figure 4: Activity diagram for payment evidence review

Check-out Settlement

The check-out activity coordinates operations and finance. It records inspection data, calculates damage fees and unpaid balances, updates deposit settlement, and releases or blocks the bed depending on the inspection result.

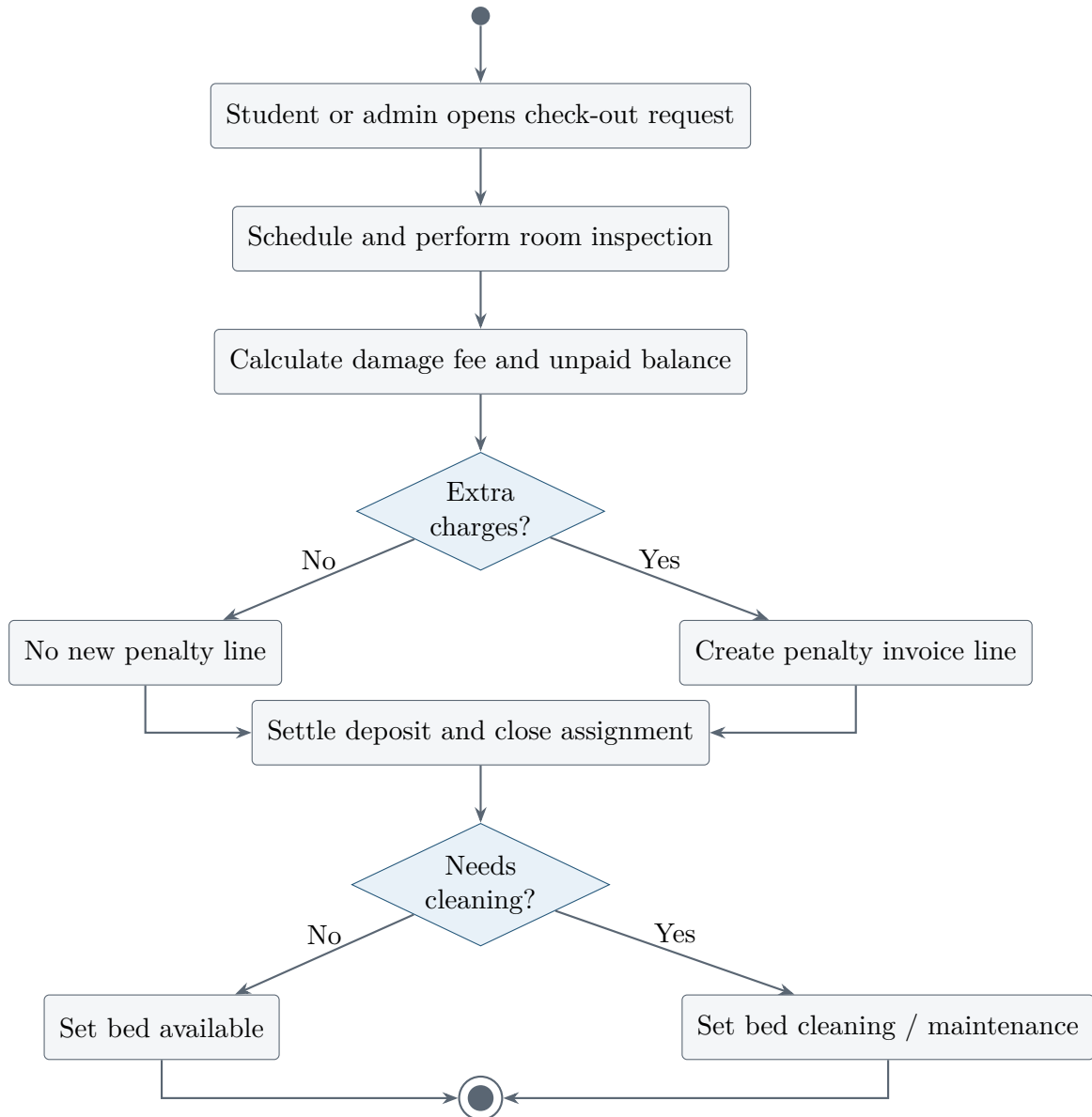


Figure 5: Activity diagram for check-out settlement

4.3 Sequence Diagrams

Sequence diagrams show the same workflows from a runtime collaboration perspective. They clarify which component owns validation, transaction control, file storage, email sending, and state updates.

Bed Reservation and Check-in

The reservation sequence shows why bed assignment must be implemented in a service transaction. The bed row is locked before the assignment is created so two admins cannot reserve the same bed concurrently.

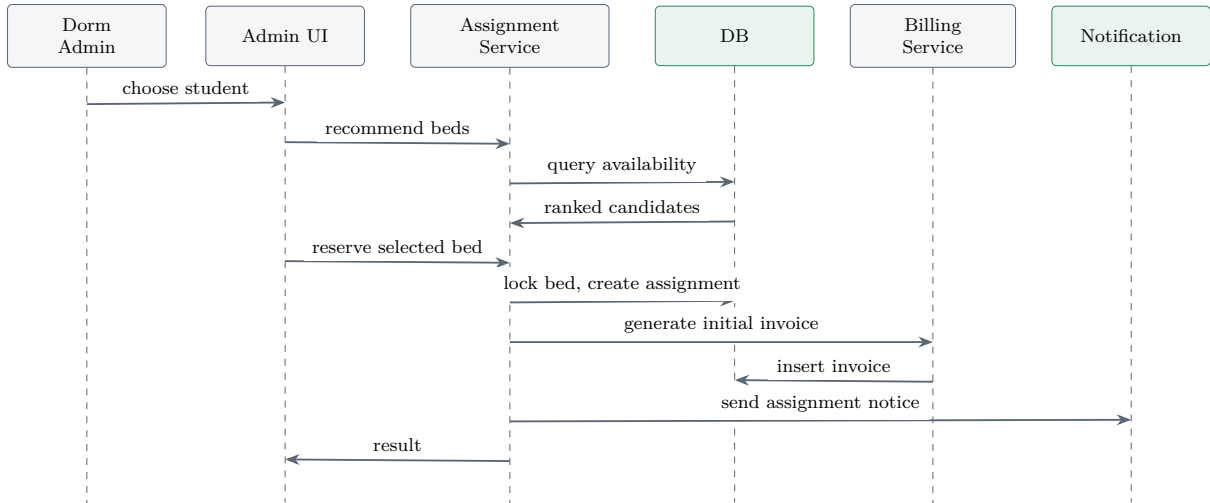


Figure 6: Sequence diagram for bed reservation and check-in preparation

Payment Evidence Review

This sequence separates file storage from database state. The system first validates invoice ownership, stores the evidence file, creates a pending payment, and then asks finance to review. Only the final finance decision updates the invoice balance.

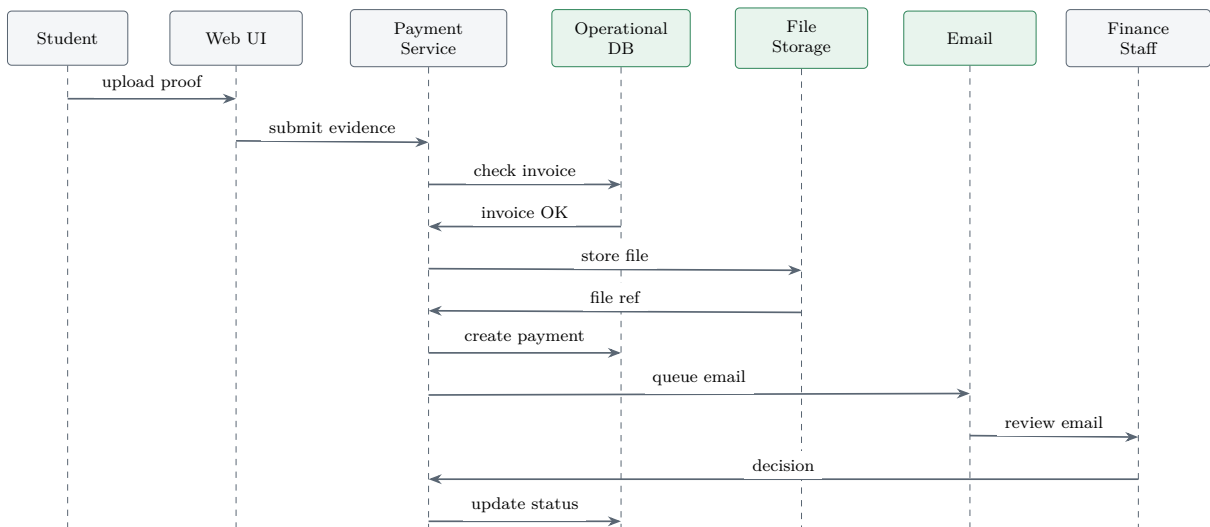


Figure 7: Sequence diagram for payment evidence review

Check-out Settlement

The check-out sequence shows that checkout is not just a bed status update. It joins inspection, billing, deposit adjustment, assignment closure, and audit logging in one transactional service operation.

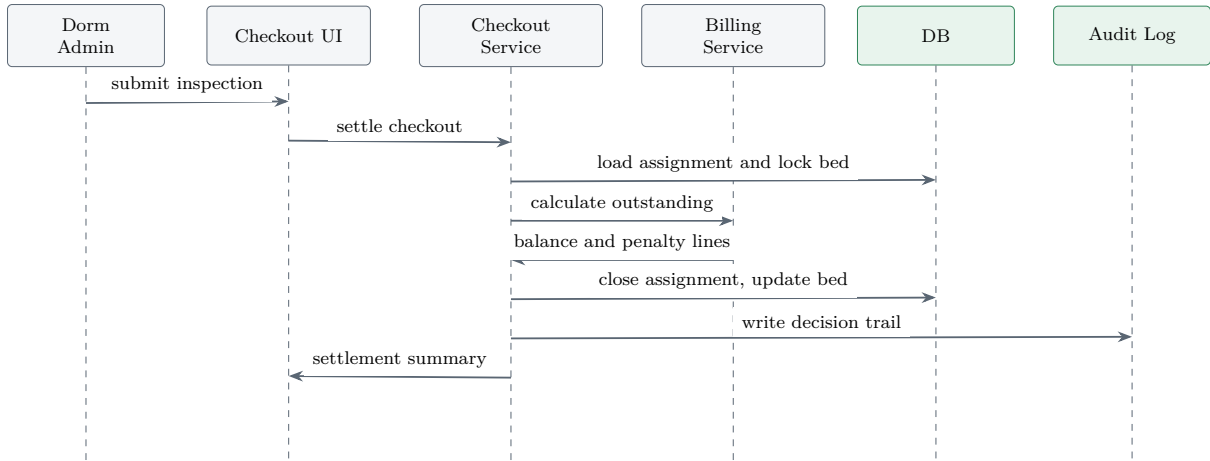


Figure 8: Sequence diagram for check-out settlement

4.4 State Diagrams

State diagrams define legal lifecycle transitions. They are the basis for validation functions such as `set_bed_status`, `confirm_payment`, and `settle_checkout`.

Bed Lifecycle

The bed lifecycle prevents double booking. Only available beds can be reserved, only reserved beds can become occupied, and check-out moves the bed either back to available or into cleaning/maintenance.

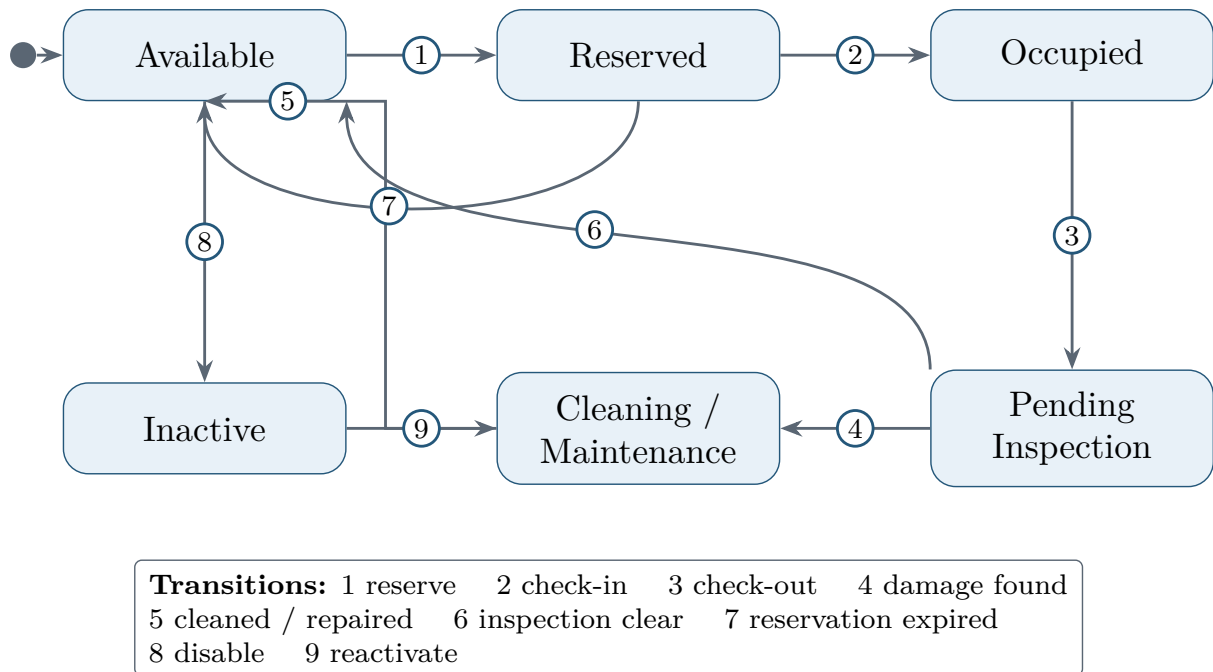


Figure 9: State diagram for a bed

Assignment Lifecycle

The assignment state controls resident history. A canceled or completed assignment remains as historical evidence, while transfer creates a new assignment record instead of overwriting the old one.

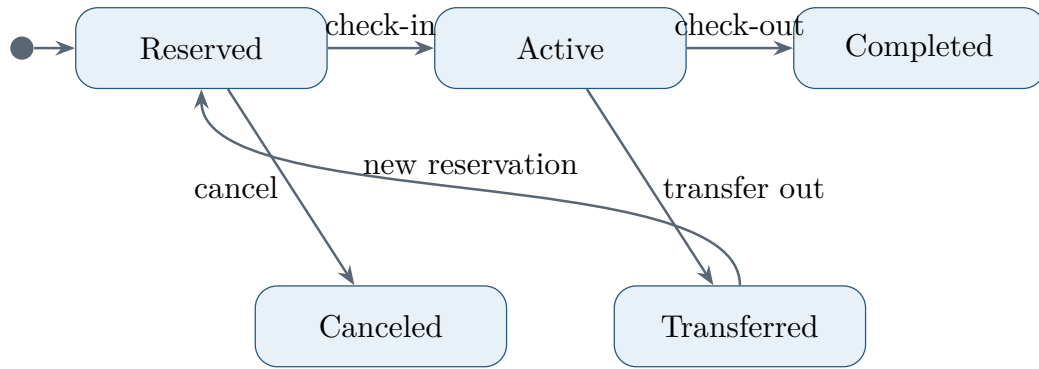


Figure 10: State diagram for an assignment

Payment Lifecycle

The payment lifecycle ensures that uploaded evidence is not treated as money until finance approves it. Confirmed payments can settle invoices; rejected payments remain visible for audit and student feedback.

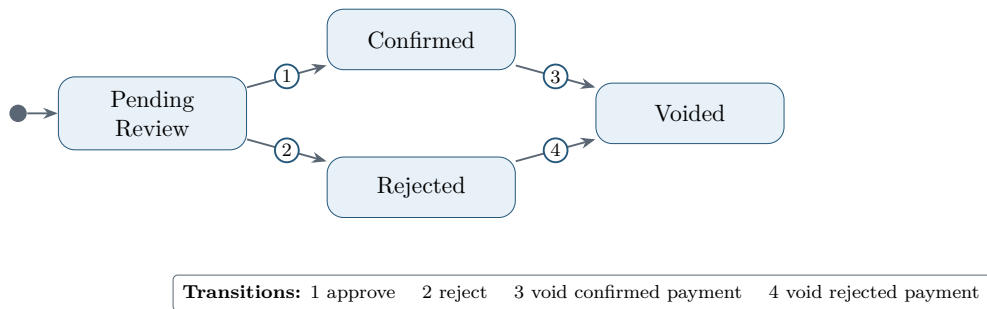


Figure 11: State diagram for a payment submission

4.5 Domain Model

The domain model shows the main conceptual objects before implementation-specific fields are introduced in the database design. It highlights three central relationships: students have assignments over time, assignments bind students to beds, and invoices are settled through reviewed payments.

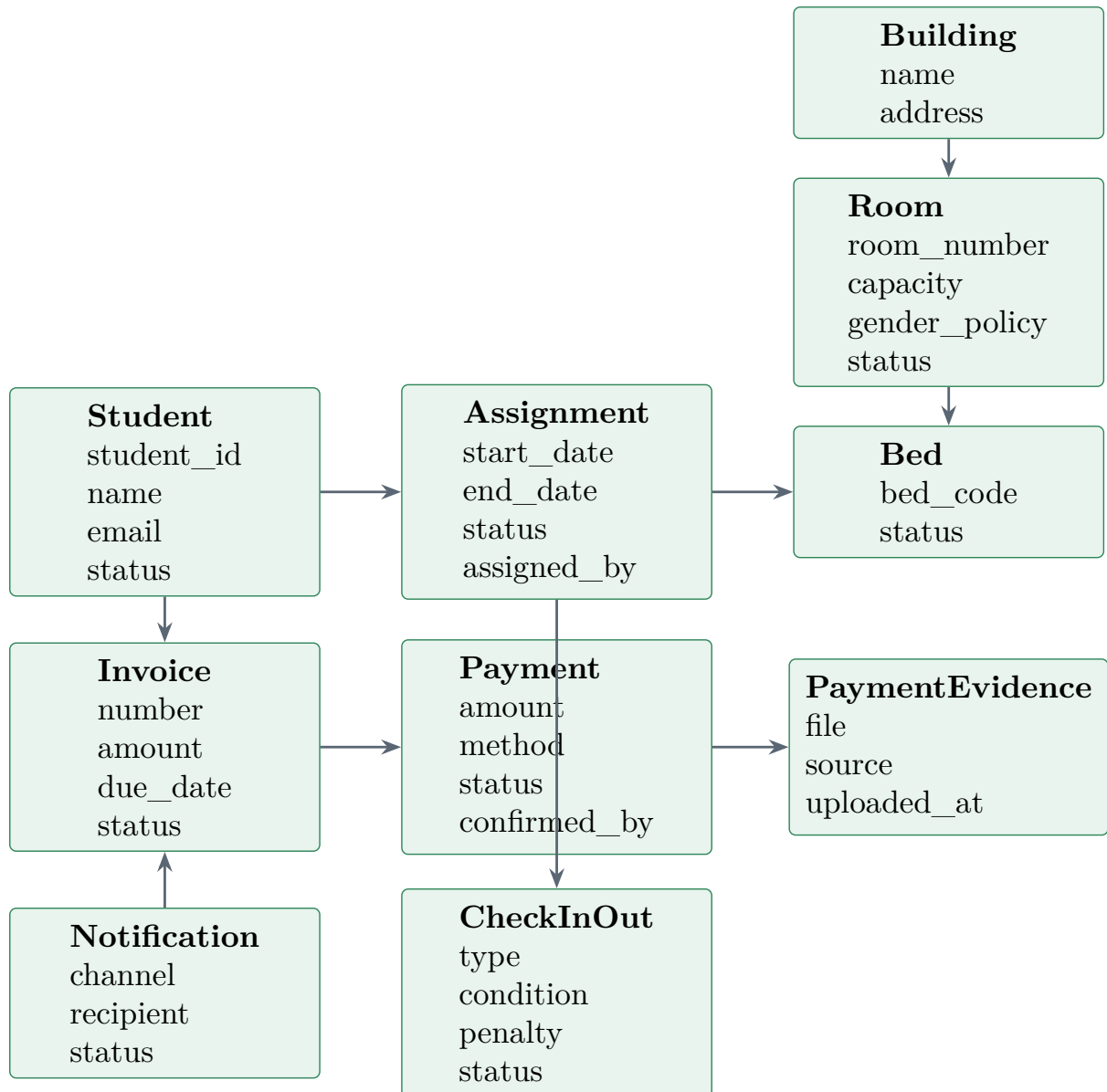


Figure 12: Domain model

The domain model deliberately omits database implementation details such as foreign-key column names. Those details are added in Section 7. The purpose here is to define the stable vocabulary used by requirements, services, tests, and user-interface labels.

4.6 Data Flow Diagram

The Level 0 DFD identifies processes, data stores, external entities, and major information flows. It complements the use case model by showing where records are created or consumed. The numbered legend keeps flow names readable while still making the data movement explicit.

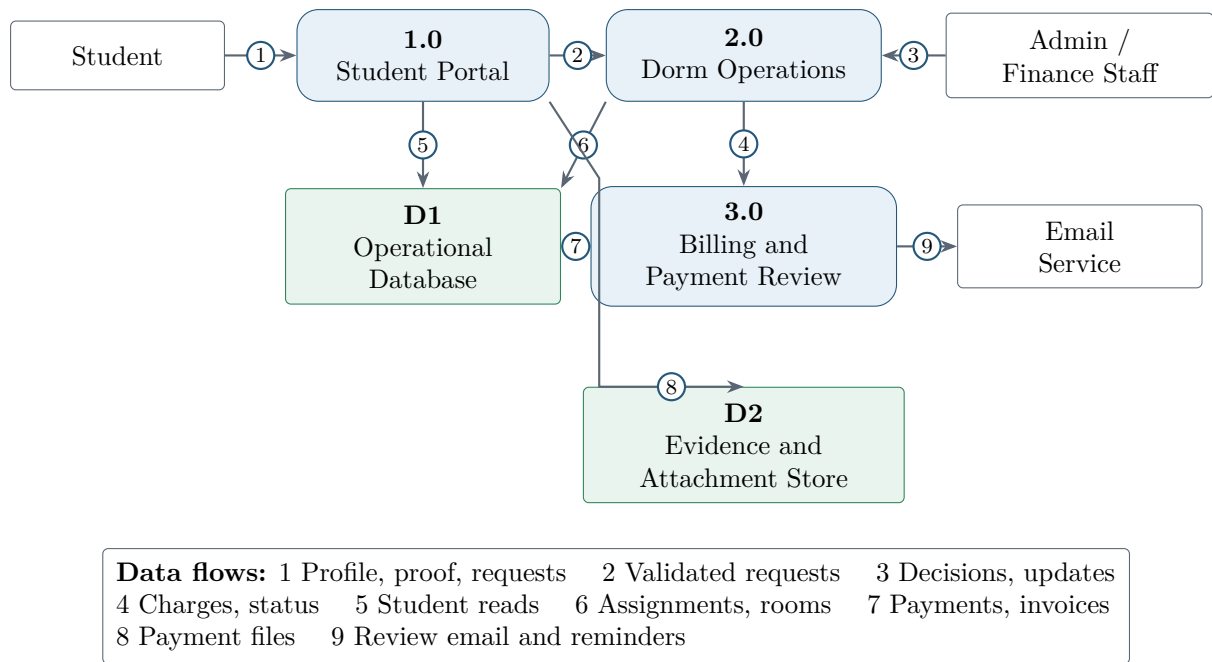


Figure 13: Level 0 data flow diagram

5 System Design Overview

5.1 Design Approach

The design transforms the analysis model into a modular Django application. Each operational area is implemented as a Django app with models, forms, views, templates, service classes, permissions, and tests. Business rules that affect correctness, such as bed assignment, payment confirmation, invoice settlement, and check-out release, are implemented in service classes instead of being embedded directly in views.

The selected implementation style is a server-rendered Django application rather than a split React front end and separate API backend. A split design would be useful for a large multi-client product, but it would add a separate build pipeline, cross-origin configuration, duplicated form validation, and more deployment moving parts for this course MVP. Django templates, forms, Bootstrap, and HTMX are sufficient for the planned demo while still allowing REST-style JSON endpoints under `/api/v1` for asynchronous widgets and future integrations.

5.2 MVP Subsystems

Subsystem	Responsibilities	Primary Users
Identity and Access	Authentication, role mapping, permission checks, user profile linkage.	All users
Resident Management	Student profiles, resident status, contacts, search, import, history.	Admin, Student
Room Inventory	Buildings, floors, rooms, beds, capacity, bed state transitions.	Admin
Assignment and Occupancy	Assignment recommendation, reservation, check-in activation, transfer, release.	Admin

Subsystem	Responsibilities	Primary Users
Billing	Invoice generation, line items, deposits, rent, fees, penalties, outstanding balance.	Finance
Payment Review	Evidence upload, pending payment queue, email summary, confirmation, rejection.	Student, Finance
Communication	Announcements, reminders, notification log.	Admin, Finance, Student
Reporting	Occupancy dashboard, finance dashboard, downloadable summaries.	Admin, Finance, Management
Demo Data	Synthetic student, room, invoice, payment, and assignment data generation.	Developers, Demo users

5.3 Planned Django App Layout

The implementation should be organised as a layered Django project. Each app owns its models, forms, views, templates, services, permissions, and tests. Shared helpers are kept small and framework-aware code should not leak into domain services.

```
dorm_system/
  config/          # settings, urls, ASGI/WSGI
  accounts/        # auth extensions, roles, permissions
  residents/       # student profiles and resident history
  rooms/           # building, floor, room, bed inventory
  assignments/     # recommendation, reservation, transfer, check-in
  billing/         # invoices, invoice lines, balances, overdue rules
  payments/        # payment evidence upload and finance review
  communications/  # announcements, reminders, notification log
  reports/         # dashboards, CSV exports, aggregate queries
  demo_data/       # seed command and synthetic fixtures
  common/          # audit log, status helpers, file validators
```

The main dependency direction is:

```
templates/forms/views -> services -> models/querysets -> database
```

Views may import services and forms. Services may import models, querysets, gateways, and audit helpers. Models must not import views or templates. This rule keeps the layered design testable.

5.4 High-Level Request Flow

1. Browser sends a request to a Django view.
2. Middleware authenticates the user session and applies security controls.
3. The view validates permissions and form or JSON input.
4. The view calls a service class for business logic.
5. The service reads and writes models inside a transaction where consistency matters.
6. The result is returned as an HTML page, HTMX partial, redirect, or JSON response.
7. Notification tasks send email after database changes are committed.

6 Architectural Design

6.1 Architecture Style

The system uses a layered MVC-style client-server architecture implemented as a modular Django monolith. The browser receives HTML for primary pages, while selected JSON endpoints provide stable contracts for dashboard widgets, recommendations, and future clients. This keeps the local demo simple without removing a later path to a separate front end.

- **Presentation layer:** Django templates, Bootstrap components, and HTMX partial updates.
- **Controller layer:** Django views and URL routes.
- **Application service layer:** assignment, billing, payment, notification, and reporting services.
- **Domain/data layer:** Django models, querysets, constraints, and PostgreSQL.
- **Integration layer:** email service, file storage, import/export, and future external systems.

6.2 Design Goals and Principles

The design follows the same analysis-to-design discipline used in the reference report: every design principle is tied to a concrete implementation mechanism. The main goal is a layered monolith that is simple enough for a project demo but structured enough to extend after the MVP.

Principle	Meaning in This System	Implementation Mechanism
Abstraction	Controllers should not know the details of assignment scoring, invoice settlement, file storage, or email delivery.	Service classes expose intention-revealing methods such as <code>reserve_bed</code> , <code>confirm_payment</code> , and <code>send_digest</code> .
Modularity	Each business capability should have one main place to change.	Separate Django apps for accounts, residents, rooms, assignments, billing, payments, notifications, reports, and demo data.
High cohesion	A module should own one operational responsibility.	Assignment logic stays in the assignment service; invoice balance logic stays in billing; payment evidence review stays in payments.
Low coupling	Modules should interact through stable calls rather than shared implementation details.	Views call services; services use model/query APIs and integration gateways; templates do not perform business decisions.

Principle	Meaning in This System	Implementation Mechanism
Information hiding	Volatile external decisions should be isolated.	Email, file storage, future payment gateways, and future university SSO are hidden behind small gateway functions.

6.3 Design Patterns Applied

The system deliberately uses a small set of patterns that fit a Django MVP. These patterns are selected because they make the implementation easier to reason about rather than because the project needs heavyweight architecture.

Pattern	Problem Solved	Use and Benefit in This System
Layered MVC / Django MTV	Page rendering, request handling, workflow rules, and persistence can become mixed together.	Templates render pages, views validate requests and permissions, services coordinate workflows, and models persist state. This keeps each layer understandable during implementation.
Service Layer	Assignment, checkout, billing, and payment review require multi-step rules and transactions.	Assignment, billing, payment, checkout, and notification services keep workflow rules out of views and templates.
Strategy	Assignment scoring may change after the MVP.	<code>AssignmentScorer</code> starts with a policy-based scorer; preference-based or roommate-matching scorers can be added later without rewriting the assignment service.
State	Beds, assignments, invoices, payments, and notifications have legal lifecycle transitions.	Status fields and service validators prevent invalid transitions such as confirming a rejected payment or occupying a maintenance bed.
Repository / Query Object	Dashboard and queue screens need repeated filtered queries.	Django managers/queriesets provide named methods such as <code>available_beds()</code> , <code>pending_payments()</code> , and <code>overdue_invoices()</code> , improving reuse and testability.
Adapter / Gateway	Email, file storage, SSO, and payment providers may differ between demo and deployment.	<code>EmailGateway</code> and <code>FileStorageGateway</code> isolate provider details so services do not depend on local disk or one SMTP implementation.

Pattern	Problem Solved	Use and Benefit in This System
Observer	Follow-up work should happen after important events without cluttering core workflows.	Payment upload, invoice overdue marking, and announcement publishing create notification events consumed by the notification service.

6.4 Architecture Diagram

The architecture diagram gives the high-level dependency rule: browser interactions enter through Django views, views call services, services coordinate domain models and gateways, and persistent state is stored in PostgreSQL or media storage. Dependencies point downward; templates and views must not contain assignment, billing, or payment-review rules.

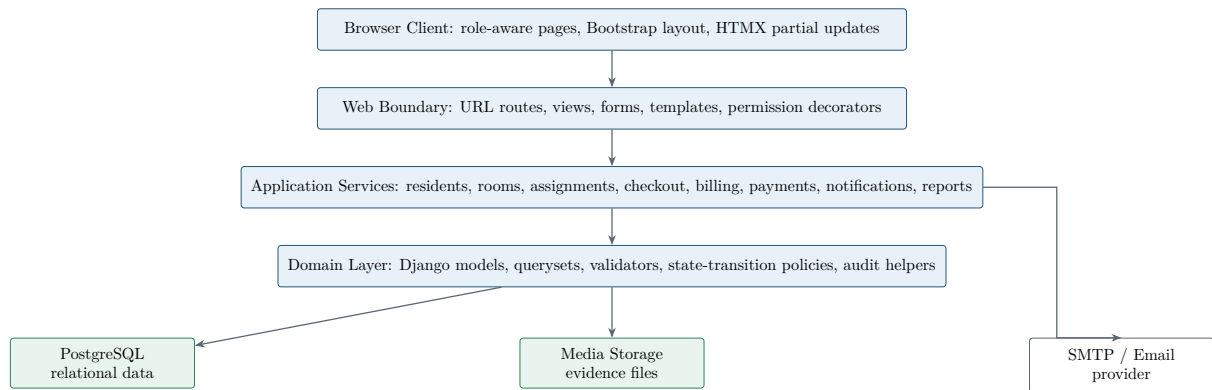


Figure 14: Layered MVC architecture

6.5 Package and Layer Decomposition

The package view turns the architecture into an implementation map. Each package corresponds to a Django app or a small shared support package. The presentation and controller packages depend on service packages; service packages depend on domain packages and infrastructure gateways. This decomposition is the main maintainability mechanism because a future payment gateway, SSO adapter, or visitor module can be added at a clear package boundary.

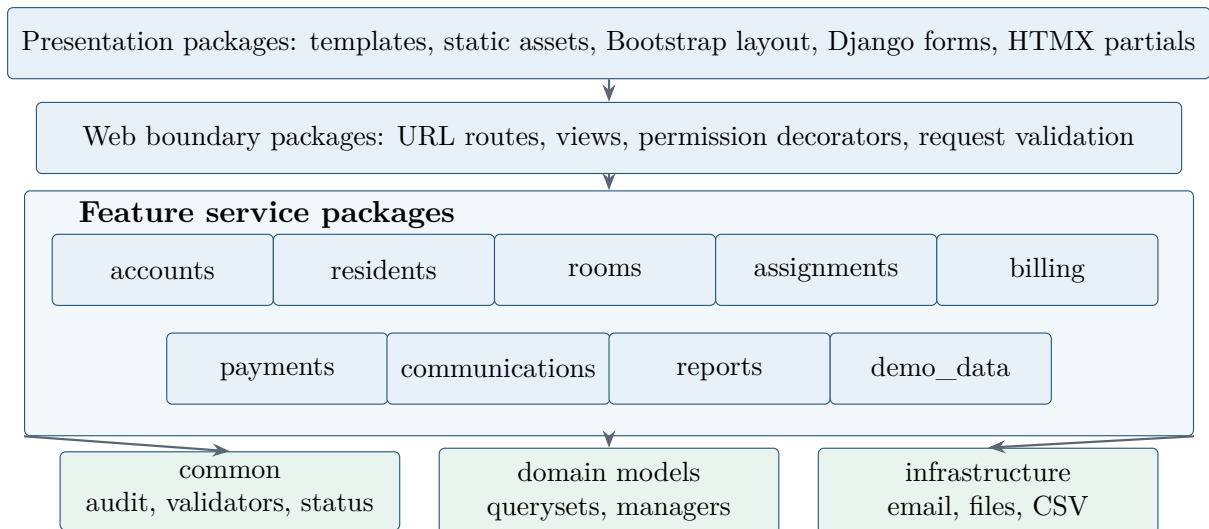


Figure 15: Package and layer decomposition

6.6 Component Diagram

The component diagram opens the server box and shows deployable or replaceable components. The important design point is that external systems are reached through gateway components. For example, `PaymentReviewService` does not know whether files are local or S3-compatible; it only calls the file-storage gateway.

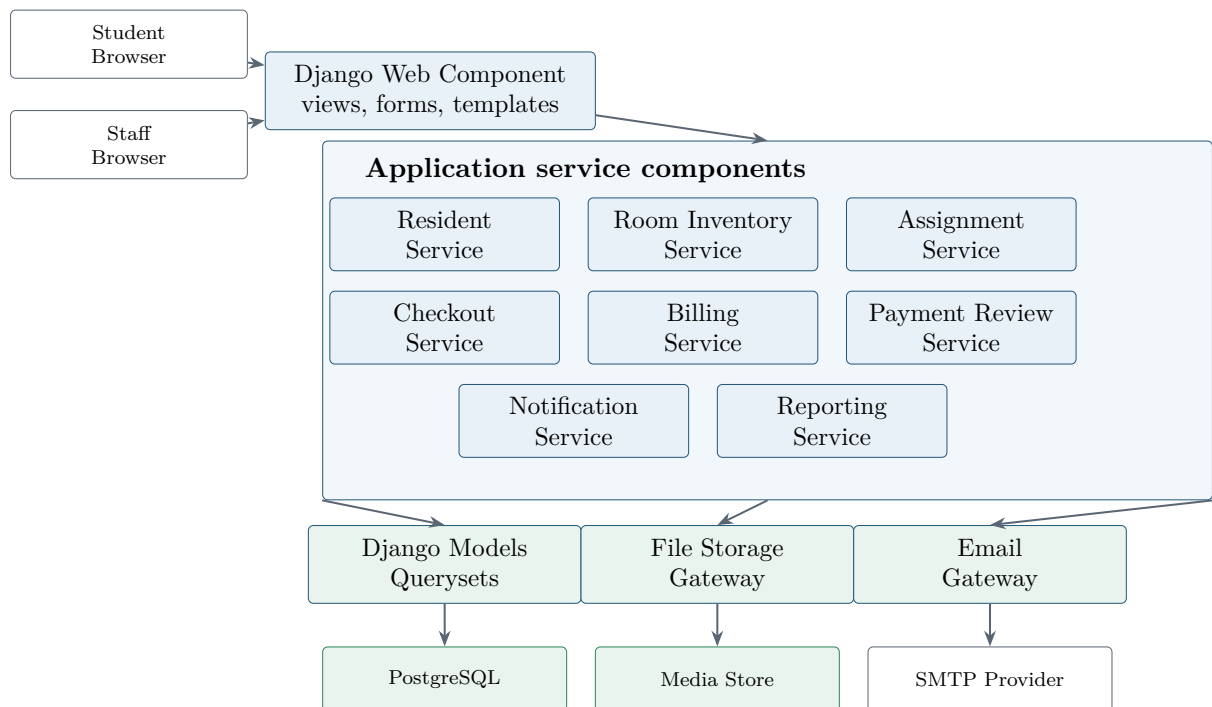


Figure 16: Component diagram

6.7 Technology Stack

Layer	Selected Technology	Rationale
Frontend	Django templates, Bootstrap, HTMX	Fast MVP development, responsive interface, server-rendered pages, simple partial updates.
Backend	Python, Django	Mature web framework with ORM, authentication, admin tools, forms, migrations, and testing support.
API	Django JSON views, optional Django REST Framework later	Keeps MVP dependencies small while preserving versioned REST contracts for integrations.
Database	PostgreSQL	Strong relational integrity, transactions, constraints, indexes, and reporting support.
Authentication	Django auth with role groups	Suitable for MVP role-based access. Can later integrate OAuth or university SSO.
Files	Local media storage for demo, S3-compatible storage for deployment	Stores payment evidence and inspection attachments.
Email	SMTP or console backend in development	Supports reminders and payment review summaries.
Deployment	Docker Compose or cloud VM/container	Repeatable local demo and simple production migration path.

This stack is intentionally conservative. Express or FastAPI with React would provide clearer separation for a public API product, but it would require more boilerplate, more testing surfaces, and an additional Node-based front-end build. For the project schedule, Django provides authentication, validation, forms, database migrations, admin utilities, and server-rendered pages in one deployable unit. The architecture still leaves an upgrade path: if a React or mobile client is added later, the `/api/v1` endpoints and service layer can become the stable backend contract.

6.8 Deployment Overview

For the demo, the system can run on one development machine using Docker Compose. The browser connects to a reverse proxy or directly to Django in development. The Django container talks to PostgreSQL, writes media files to a persistent volume, and sends email through either the console backend or an SMTP provider. This topology is practical for a local presentation because it avoids a separate frontend server and can be reset with migrations plus synthetic seed data. The same topology can move to a low-cost VM or container host by replacing the local media volume with S3-compatible storage, adding managed PostgreSQL if needed, and terminating TLS at the reverse proxy.

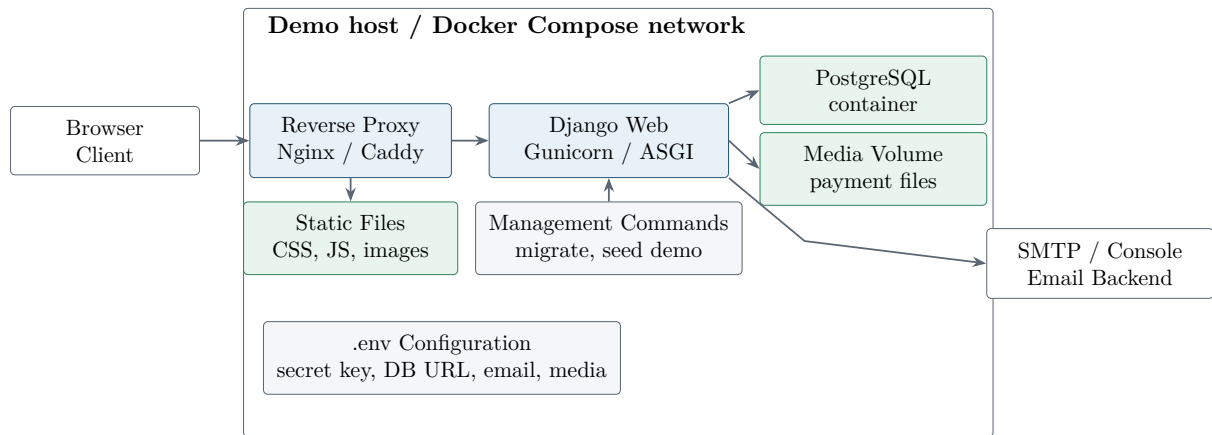


Figure 17: Deployment overview

7 Data / Class Design

7.1 Class Diagram

The class design is presented as one integrated UML class diagram generated from Mermaid source. It follows the convention guide: each class has a name, private attributes, and public operations; solid lines represent associations, filled diamonds represent composition, and dashed arrows represent dependencies. Multiplicities are placed near both class ends, using values such as 1, 0..1, 0..*, and 1..*. Primary-key and foreign-key columns are intentionally reserved for the ERD and database schema sections.

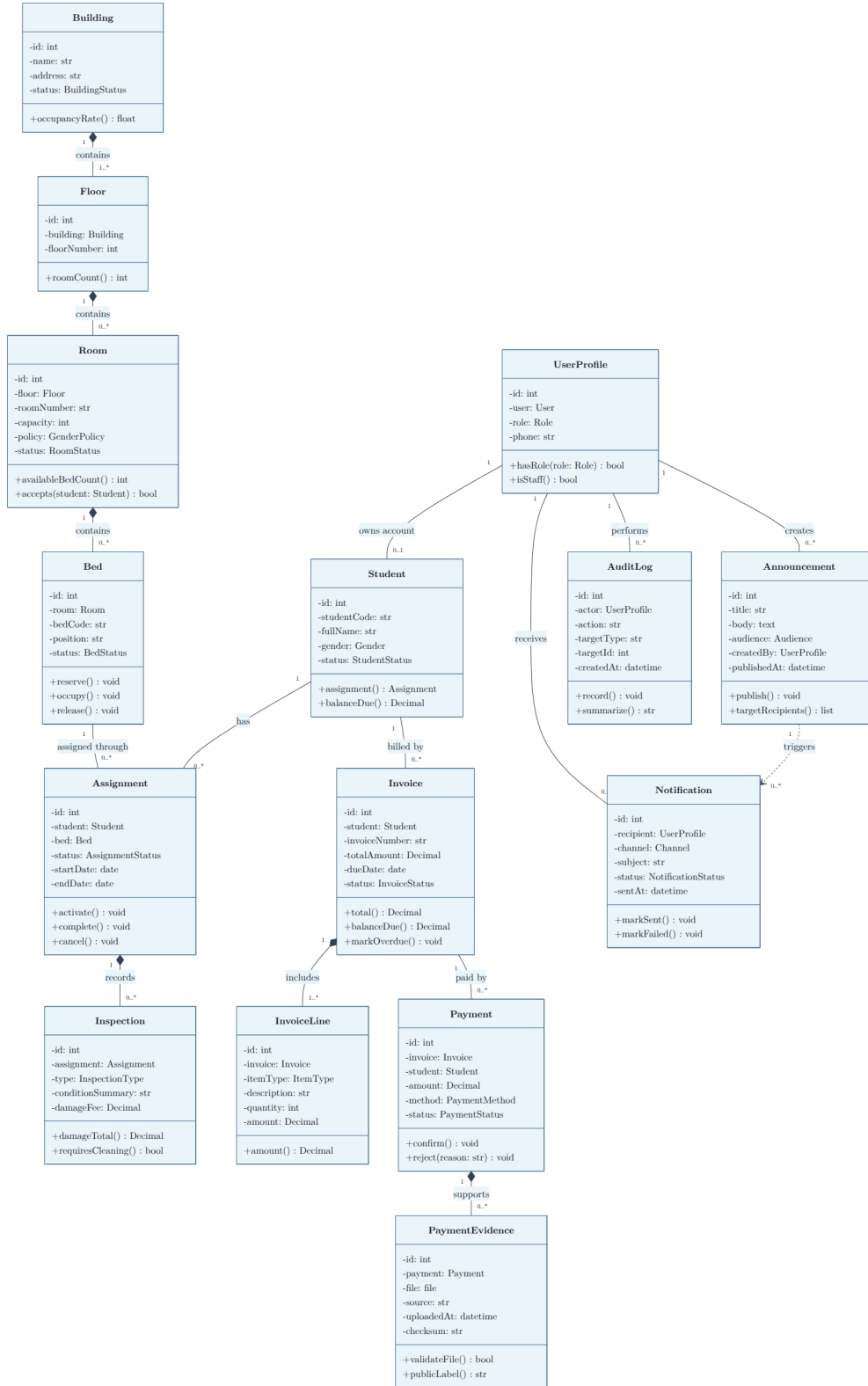


Figure 18: Unified UML class diagram for the dormitory management system

7.2 Database Schema

Table	Key Fields	Notes
auth_user	id, username, email, password, is_active	Django authentication table.
user_profile	id, user_id, role, phone	Role and profile extension.
student	id, user_id, student_code, full_name, gender, email, phone, status	Student master record.
building	id, name, address, status	Dorm building.
floor	id, building_id, floor_number	Optional physical grouping.
room	id, floor_id, room_number, capacity, gender_policy, status	Room inventory and policy.
bed	id, room_id, bed_code, position, status	Assignable unit.
assignment	id, student_id, bed_id, status, start_date, end_date, assigned_by_id, reason	Occupancy history.
inspection	id, assignment_id, type, condition_summary, damage_fee, notes, inspected_by_id	Check-in and check-out evidence.
invoice	id, student_id, invoice_number, total_amount, due_date, status	Billing header.
invoice_line	id, invoice_id, item_type, description, quantity, unit_price, amount	Rent, deposit, fee, penalty.
payment	id, invoice_id, student_id, amount, method, status, submitted_at, reviewed_by_id	Payment review state.
payment_evidence	id, payment_id, file, source, uploaded_at, checksum	Uploaded proof files.
announcement	id, title, body, audience, created_by_id, published_at	Broadcast notices.
notification	id, recipient_id, channel, subject, body, status, sent_at	Email and in-app log.
audit_log	id, actor_id, action, target_type, target_id, old_value, new_value, created_at	Audit trail.

7.3 Database Constraints and Indexes

The implementation should enforce the most important business rules in the database as well as in services.

Constraint / Index	Purpose
Unique student code	Prevent duplicate student master records.
Unique room per floor	Prevent duplicate room numbers on the same floor.
Unique bed per room	Prevent duplicate bed codes in one room.

Constraint / Index	Purpose
Partial unique index on active assignment per student	Enforce at most one active or reserved assignment for a student.
Partial unique index on active assignment per bed	Enforce at most one active or reserved assignment for a bed.
Unique invoice number	Keep invoice references stable for payment review.
Indexes on status and date fields	Support dashboards, overdue detection, and pending payment queues.
Foreign keys with restricted or protected delete where history matters	Preserve auditability for invoices, assignments, payments, and inspections.

7.4 Entity Relationships

The ERD is presented as one generated crow-foot diagram. It keeps the database notation separate from the UML class diagram: entities are tables, attributes are database columns, PK/FK markers are shown inside the entities, and cardinality is shown on the relationship ends. Conceptually, students and beds are **N:M over time**; the implementation resolves this through the `assignment` associative entity.

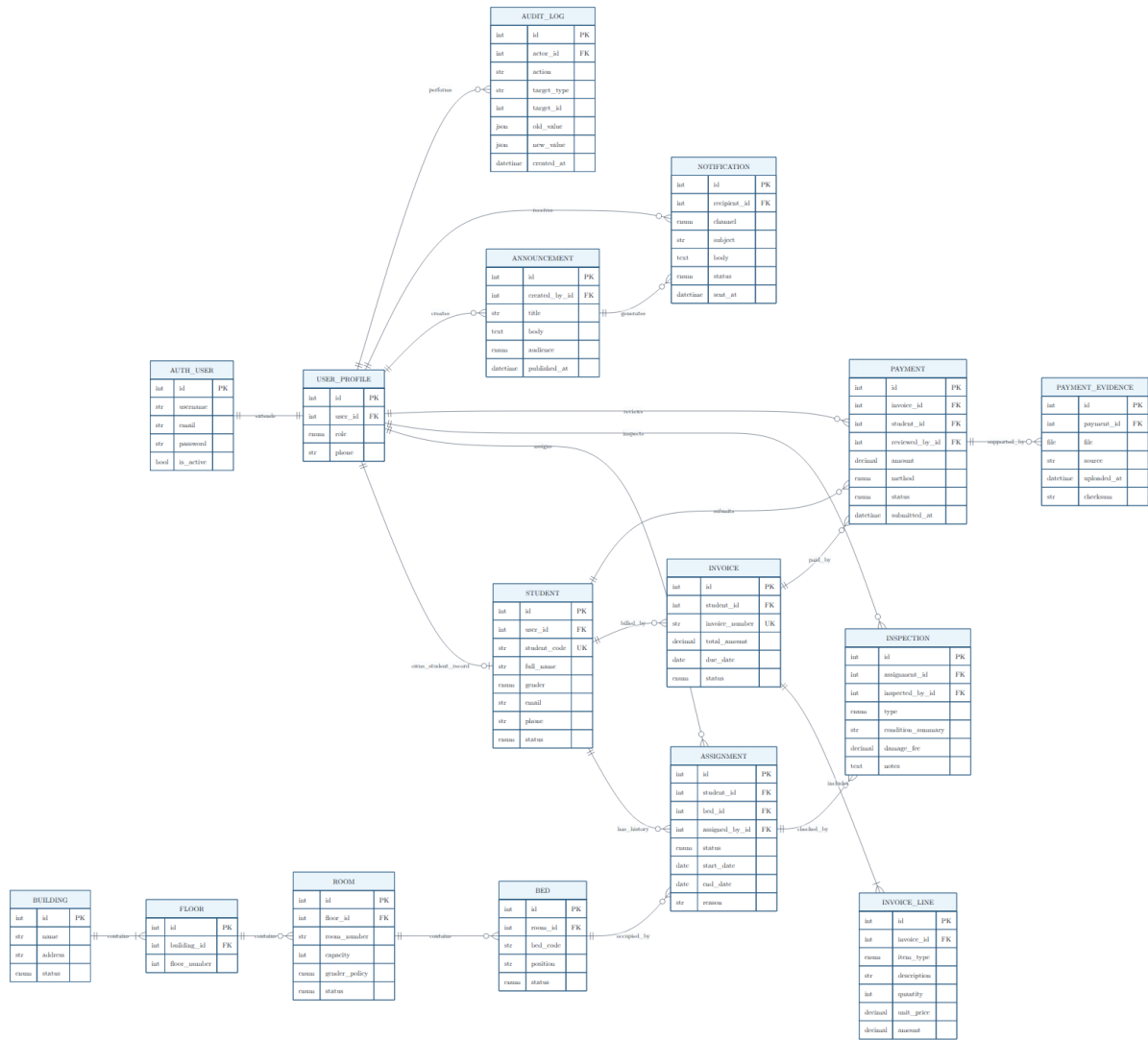


Figure 19: Unified entity relationship diagram for the dormitory management system

The crow-foot ERD translates directly into database constraints and Django model fields. Inventory tables describe physical capacity, assignment tables describe occupancy history, billing tables describe receivables and proof review, and communication/audit tables preserve operational evidence.

Relationship	Cardinality	Implementation note
UserProfile–Student	1:0..1	Only student accounts have a linked Student record; staff profiles do not.
Building–Floor	1:1..*	A building has one or more floors.
Floor–Room, Room–Bed	1:0..*	Floors may have many rooms and rooms may have many beds; empty setup states are valid.
Student–Assignment and Bed–Assignment	1:0..*	Assignment resolves the conceptual Student–Bed many-to-many relationship over time.

Relationship	Cardinality	Implementation note
Assignment–Inspection	1:0..*	A stay can have zero, check-in, check-out, or follow-up inspection records.
Student–Invoice and Invoice–Payment	1:0..*	Students may have many invoices, and an invoice may have zero or more payment attempts.
Invoice–InvoiceLine	1:1..*	Every issued invoice contains at least one charge line.
Payment–PaymentEvidence	1:0..*	Some payment methods require uploaded evidence; manual or cash records may not.
UserProfile–Announcement, UserProfile–Notification, UserProfile–AuditLog	1:0..*	A user can create announcements, receive notifications, and appear as the actor in audit entries.
Announcement–Notification	1:0..*	Publishing a broadcast can generate recipient notifications.

7.5 Data Dictionary

Entity	Field	Type	Description
Student	student_code	varchar	Unique university student identifier.
Student	status	enum	applicant, active, inactive, checked_out, suspended.
Room	gender_policy	enum	male, female, mixed, unspecified. Used by assignment rules.
Room	capacity	integer	Number of beds that should exist in the room.
Bed	status	enum	available, reserved, occupied, cleaning, maintenance, inactive.
Assignment	status	enum	reserved, active, transferred, completed, canceled.
Assignment	start_date	date	First effective residence date.
Invoice	status	enum	draft, issued, partially_paid, paid, overdue, canceled.
InvoiceLine	item_type	enum	rent, deposit, fee, penalty, adjustment.
Payment	status	enum	pending_review, confirmed, rejected, voided.
Payment Evidence	source	varchar	Upload channel or evidence source, such as student portal or admin import.
Notification	status	enum	queued, sent, failed, read.

Entity	Field	Type	Description
Audit Log	old_value / new_value	json	Snapshot of important state before and after change.

7.6 Important Data Structures

- **AssignmentCandidate:** in-memory object containing bed id, room id, current occupancy, policy match, preference score, and total score.
- **InvoiceBalance:** computed structure containing invoice total, confirmed payments, pending payments, rejected payments, and outstanding amount.
- **DashboardSummary:** aggregate counts for available beds, occupied beds, overdue invoices, pending payments, and check-out inspections.
- **PaymentReviewDigest:** grouped pending payment records used to generate finance review emails.

8 Interface Design

8.1 User Interface Overview

The user interface is divided into role-focused areas. Students see their profile, assignment, invoices, payment upload form, announcements, and check-out request. Dorm admins see resident search, room inventory, assignment tools, check-in/check-out queues, announcements, and occupancy reports. Finance staff see invoice generation, payment review, overdue balances, deposits, and financial reports.

The interface figures in this report are placeholder wireframes used to define layout intent before implementation. After the working demo is built, these placeholders should be replaced with real screenshots from the running system so the final report accurately reflects the implemented user interface.

8.2 Information Architecture and Sitemap

The interface uses a persistent shell with role-aware navigation. After sign-in, each role lands on the page that best matches its daily work: students land on their resident portal, dorm admins land on the operations dashboard, and finance staff land on billing/payment review. The sitemap keeps primary workflows within one navigation click and avoids hiding core work behind deep menus.

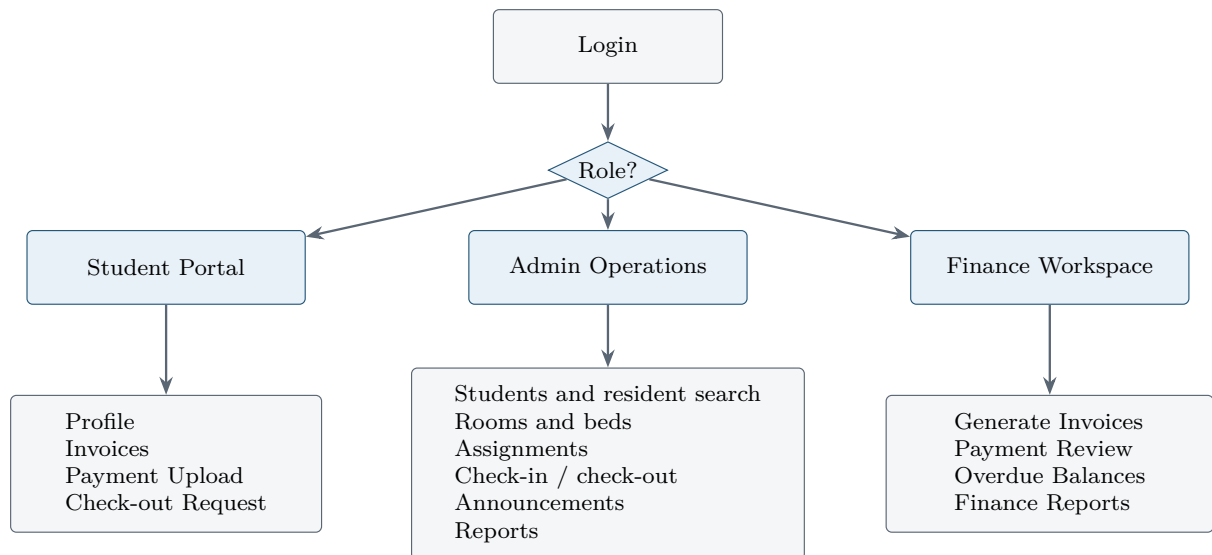


Figure 20: Role-aware sitemap

8.3 Core User Flow

The user-flow diagram focuses on the workflows most important to the demo. It shows the role gate after authentication and the main task loops for student payment upload, admin assignment/check-in, admin check-out, and finance confirmation. Each path ends with a status update that is reflected on dashboards and reports.

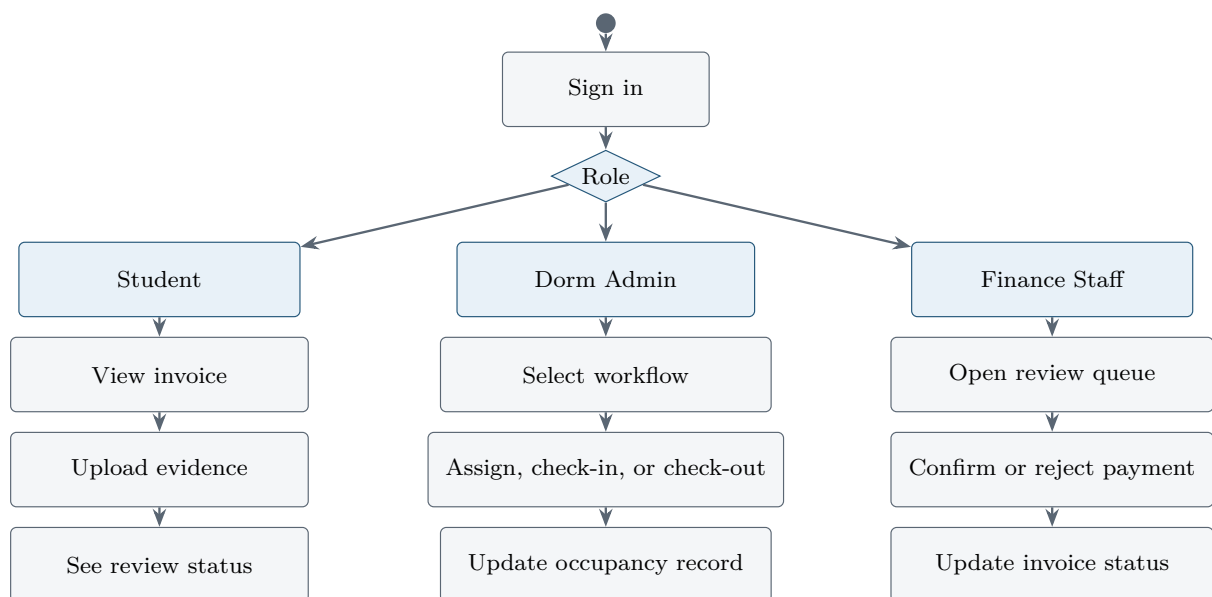


Figure 21: Core user flow

8.4 Front-end Component Structure

The front end is server-rendered rather than a single-page application, but it still has a component structure. Reusable Django templates, forms, and HTMX partials act as the front-end components. This structure should be preserved during implementation so that dashboard cards, tables, filters, modals, and status badges are not duplicated across pages. A later React front end could reuse the same service-backed `/api/v1` endpoints, but it is not required for the local demo.

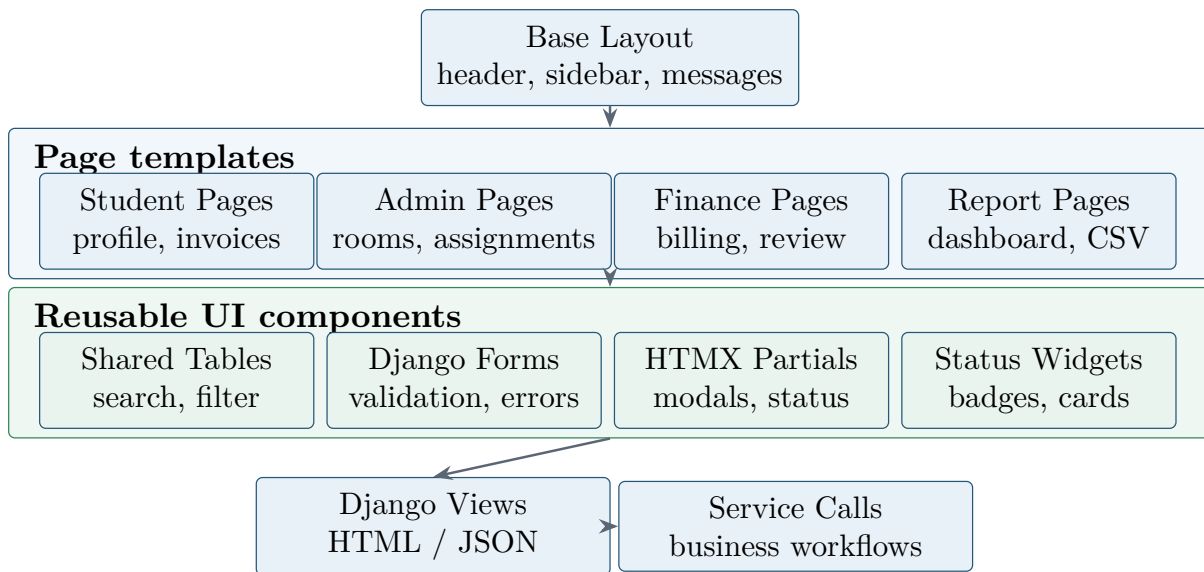


Figure 22: Front-end component structure

8.5 Screenshot Replacement Plan

Because the demo has not been implemented yet, this report uses placeholder wireframes instead of invented screenshots. After implementation, the final report should replace the placeholders with real screenshots for the login page, admin dashboard, room assignment screen, student payment upload screen, finance payment review screen, and check-out settlement screen.

8.6 Dashboard Placeholder

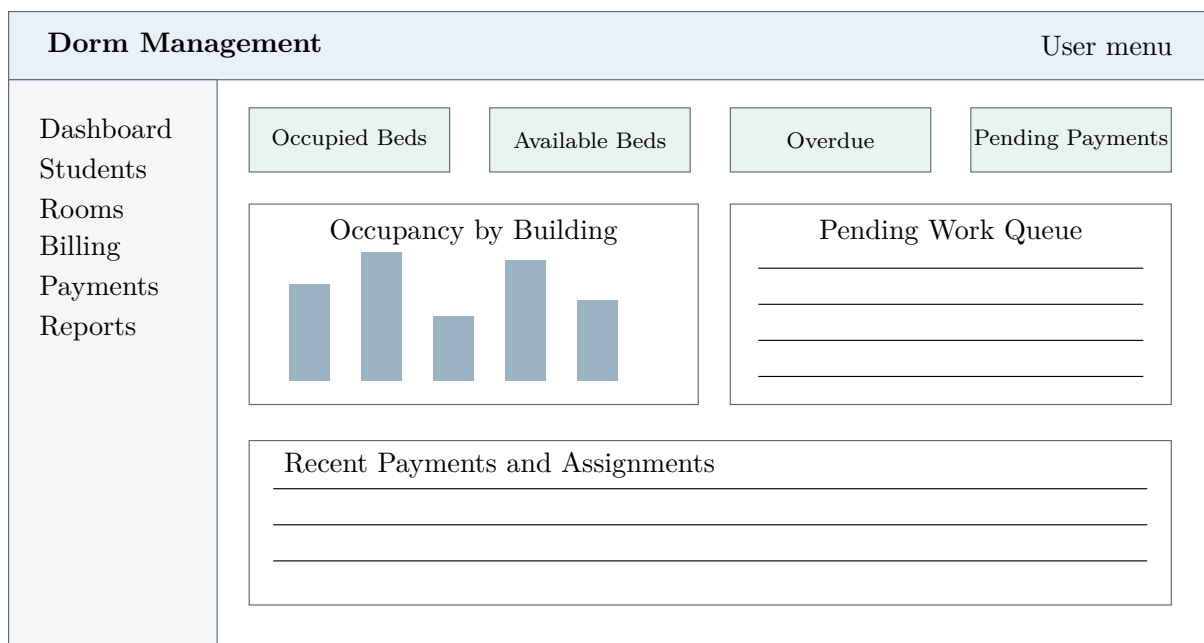


Figure 23: Administrative dashboard placeholder wireframe

8.7 Payment Review Placeholder

Payment Review	
Search invoice, student, amount, status	
INV-2026-001 Nguyen A 1.5M VND Evidence: bank-transfer-001.jpg, uploaded from student portal	[View] [Confirm] [Reject]
INV-2026-002 Tran B 1.5M VND Evidence: payment-slip.pdf, uploaded from student portal	[View] [Confirm] [Reject]
INV-2026-003 Le C 750K VND Evidence: admin-import.csv row 15	[View] [Confirm] [Reject]

Figure 24: Payment review placeholder wireframe

8.8 API Design

The MVP is primarily server-rendered, so user-facing navigation uses HTML routes such as `/students/`, `/rooms/`, and `/payments/review/`. The API contract is documented separately as REST-style JSON endpoints for asynchronous widgets, dashboard data, recommendations, and future integration. The API follows the project convention: plural resource names, kebab-case for multi-word resources, JSON bodies, consistent status codes, and versioned URLs under `/api/v1`. All endpoints require authentication unless explicitly marked public.

Method	Endpoint	Purpose	Role
GET	<code>/api/v1/dashboard-summary</code>	Dashboard counts and queue summaries.	Staff
GET	<code>/api/v1/students</code>	Search and paginate students.	Admin
POST	<code>/api/v1/students</code>	Create a student record.	Admin
GET	<code>/api/v1/rooms</code>	List rooms, filters, and occupancy.	Admin
GET	<code>/api/v1/beds?status=available</code>	List available beds.	Admin
POST	<code>/api/v1/assignment-recommendations</code>	Return ranked beds for a student.	Admin
POST	<code>/api/v1/assignments</code>	Reserve or activate a bed assignment.	Admin
POST	<code>/api/v1/assignments/{assignmentId}/check-in</code>	Finalize check-in.	Admin
POST	<code>/api/v1/assignments/{assignmentId}/check-out</code>	Complete check-out settlement.	Admin
POST	<code>/api/v1/invoices/batches</code>	Generate recurring invoices.	Finance

Method	Endpoint	Purpose	Role
GET	/api/v1/students/me/invoices	Student invoice list.	Student
POST	/api/v1/payments	Upload payment evidence.	Student
POST	/api/v1/payments/{paymentId}/confirm	Confirm a pending payment.	Finance
POST	/api/v1/payments/{paymentId}/reject	Reject a pending payment.	Finance
POST	/api/v1/announcements	Create announcement.	Admin
GET	/api/v1/reports/occupancy	Return occupancy report data or CSV.	Staff

Responses use a consistent envelope:

```
{
  "success": true,
  "message": "Operation completed.",
  "data": {}
}
```

Validation errors return 400 Bad Request or 422 Unprocessable Entity; authorization failures return 401 Unauthorized or 403 Forbidden; business conflicts such as double booking return 409 Conflict.

8.9 Representative API Payloads

The final implementation may render most interactions as HTML, but the following JSON shapes define the data contract for the workflows that benefit from asynchronous responses.

Assignment recommendation request and response:

POST /api/v1/assignment-recommendations

```
{
  "studentId": 42,
  "preferences": {
    "buildingId": 2,
    "floorPreference": "low",
    "roomType": "shared"
  },
  "limit": 5
}
```

200 OK

```
{
  "success": true,
  "message": "Recommendations generated.",
  "data": {
    "studentId": 42,
    "candidates": [
      {
        "bedId": 311,
        "building": "A",

```

```

    "room": "A-203",
    "bedCode": "A-203-B",
    "availableBedsInRoom": 1,
    "policyMatch": true,
    "score": 86,
    "reasons": ["gender policy match", "preferred building"]
  }
]
}
}

```

Payment evidence upload result:

```

POST /api/v1/payments
Content-Type: multipart/form-data
fields: invoiceId, amount, evidenceFiles[]

201 Created
{
  "success": true,
  "message": "Payment evidence submitted for finance review.",
  "data": {
    "paymentId": 88,
    "invoiceId": 1204,
    "status": "pending_review",
    "evidenceCount": 2
  }
}

```

8.10 Input and Output Formats

- HTML forms are used for most CRUD workflows.
- JSON responses are used for recommendation results, dashboard widgets, and HTMX status responses.
- CSV import/export is used for student imports and report downloads.
- Uploaded payment evidence accepts common image and PDF formats with file size limits.
- Email messages are generated as plain text and HTML summaries for reminders and payment review digests.

8.11 External Interfaces

- **Email service:** sends invoice reminders, payment review summaries, and announcements.
- **File storage:** stores payment evidence and inspection attachments.
- **Future payment gateway:** can replace or supplement manual payment evidence review.
- **Future university SSO:** can replace local user authentication while preserving role permissions.

9 Component-Level / Detailed Design

9.1 Service Method Map

The service layer is the implementation boundary for workflow logic. The following table lists the main service classes that should exist in code and the method names the implementation can start from.

Service Class	Core Methods	Primary Dependencies
IdentityService	require_role, can_view_student, get_profile	Django auth, UserProfile, Student
ResidentService	create_student, import_students, get_resident_history	Student, Assignment, Invoice
RoomInventoryService	create_room, sync_beds, set_bed_status	Building, Floor, Room, Bed
AssignmentService	recommend_beds, reserve_bed, activate_assignment, transfer_student	Student, Bed, Assignment, AuditLog
CheckoutService	finalize_checkin, settle_checkout, calculate_damage_fee	Assignment, Inspection, InvoiceLine
BillingService	generate_initial_invoice, generate_recurring_rent, calculate_invoice_balance	Invoice, InvoiceLine, Payment
PaymentReviewService	submit_payment_evidence, send_review_digest, confirm_payment, reject_payment	Payment, PaymentEvidence, FileStorage, NotificationService
NotificationService	send_announcement, send_invoice_reminder, record_notification	Announcement, Notification, EmailGateway
ReportingService	dashboard_summary, occupancy_report, finance_report	Querysets, aggregates, CSV exporter

9.2 Identity and Access Module

Responsibilities: authenticate users, map users to roles, protect views, and expose role-aware navigation.

Key methods and functions:

- `require_role(user, roles)`: checks whether the current user has one of the allowed roles.
- `get_profile(user)`: returns profile and role information.
- `can_view_student(user, student)`: enforces student self-access and staff access.

Dependencies: Django auth, UserProfile, Student.

9.3 Resident Management Module

Responsibilities: manage student records, contact details, status, and resident history.

Key methods and functions:

- `create_student(data)`: validates uniqueness and creates a student record.
- `import_students(csv_file)`: validates and imports student data from CSV.
- `get_resident_history(student)`: returns assignments, invoices, and check-in/check-out records.

Internal logic: student status changes are controlled by workflow events. For example, check-in can change a student from applicant to active, while completed check-out can change active to checked_out.

9.4 Room Inventory Module

Responsibilities: manage buildings, floors, rooms, beds, capacity, and bed lifecycle state.

Key methods and functions:

- `create_room(floor, room_number, capacity, policy)`.
- `sync_beds(room, capacity)`: creates missing bed records when capacity changes.
- `set_bed_status(bed, status, reason)`: validates legal transitions and logs changes.

Dependencies: Building, Floor, Room, Bed, AuditLog.

9.5 Assignment Module

Responsibilities: recommend beds, reserve beds, activate assignments, transfer students, and maintain occupancy history.

Room assignment pseudocode:

```
function recommend_beds(student, preferences, limit):
    candidates = query beds where status = "available"
    candidates = exclude rooms with inactive or maintenance status

    for each candidate in candidates:
        score = 0
        if room gender_policy matches student gender:
            score += 40
        else if room gender_policy is "mixed" or "unspecified":
            score += 15
        else:
            continue

        score += capacity_score(room.available_bed_count)
        score += preference_score(candidate, preferences)
        score -= distance_penalty(candidate, preferences)

        candidate.score = score

    sort candidates by score descending, building, room_number, bed_code
    return first limit candidates
```

```

function reserve_bed(student, bed, actor):
    begin transaction
        lock selected bed row
        if bed.status is not "available":
            raise AssignmentConflict
        cancel previous active reservation for student if any
        create assignment with status = "reserved"
        set bed.status = "reserved"
        write audit log
    commit transaction

```

Consistency rules:

- A student may have at most one active or reserved assignment.
- A bed may have at most one active or reserved assignment.
- Reservation and activation must lock the bed row to avoid double booking.
- Manual overrides require an admin reason.

9.6 Check-In / Check-Out Module

Responsibilities: finalize move-in, record handover condition, process departure, record inspection, calculate penalties, release bed inventory, and settle deposits.

Check-out pseudocode:

```

function settle_checkout(assignment, inspection_data, actor):
    begin transaction
        assert assignment.status == "active"
        create checkout inspection from inspection_data
        penalty = calculate_damage_fee(inspection_data)
        unpaid = calculate_outstanding_balance(assignment.student)
        if penalty > 0:
            create penalty invoice line
        update deposit settlement using penalty and unpaid amount
        assignment.status = "completed"
        assignment.end_date = today
        if inspection requires cleaning:
            bed.status = "cleaning"
        else:
            bed.status = "available"
            student.status = "checked_out"
        write audit log
    commit transaction

```

9.7 Billing Module

Responsibilities: generate invoices, invoice lines, deposits, fees, rent cycles, penalties, and balances.

Key methods and functions:

- generate_initial_invoice(student, assignment).

- `generate_recurring_rent(period).`
- `calculate_invoice_balance(invoice).`
- `mark_overdue_invoices(today).`

Internal logic: invoice status is derived from due date and confirmed payment total. Pending payments do not reduce outstanding balance until confirmed.

9.8 Payment Review Module

Responsibilities: accept payment evidence, group pending submissions, send review emails, confirm or reject payments, update invoice status, and log decisions.

Payment evidence aggregation pseudocode:

```
function submit_payment_evidence(invoice, amount, files, student):
    begin transaction
        assert invoice belongs to student
        payment = create payment(status="pending_review", amount=amount)
        for file in files:
            validate file type and size
            store file and create payment evidence
            enqueue payment_review_digest(payment)
        commit transaction

function send_payment_review_digest():
    pending = query payments where status = "pending_review"
        and review_email_sent = false
    grouped = group pending by finance recipient or dorm building
    for each group in grouped:
        render email with invoice, student, amount, and evidence links
        send email
        mark payments as review_email_sent

function confirm_payment(payment, actor):
    begin transaction
        lock payment and invoice
        assert actor has finance permission
        payment.status = "confirmed"
        payment.reviewed_by = actor
        update invoice status from confirmed payment total
        write audit log
    commit transaction
```

9.9 Notification Module

Responsibilities: send announcements, invoice reminders, overdue notices, and payment review emails.

Key methods and functions:

- `send_announcement(announcement).`
- `send_invoice_reminder(invoice).`
- `send_payment_review_digest(payments).`

- `record_notification(recipient, channel, status)`.

9.10 Reporting Module

Responsibilities: produce dashboard metrics and exportable reports.

Example dashboard queries:

- Count beds by status and building.
- Count active students and pending check-ins.
- Sum outstanding invoice amounts by due-date bucket.
- List pending payment reviews with age in days.

9.11 Demo Data Module

Responsibilities: seed a realistic demo dataset that supports the presentation scenario.

Generated data:

- Buildings, floors, rooms, and beds with mixed availability.
- Student accounts with active, applicant, checked-out, and suspended statuses.
- Assignments in reserved, active, and completed states.
- Invoices with paid, overdue, partially paid, and pending review statuses.
- Payment evidence records and announcement examples.

10 Quality Attributes and Design Rationale

Attribute	Design Support	Priority
Maintainability	Django apps separate identity, residents, rooms, assignments, billing, payments, notifications, and reports. Service classes keep workflow rules out of views.	High
Scalability	PostgreSQL indexes support larger student and invoice tables. The architecture can move email sending to background jobs later.	Medium
Security	Authentication, role groups, permission checks, file validation, audit logs, and restricted student self-access protect sensitive data.	High
Reliability	Assignment and payment operations use transactions and row locks for critical state changes.	High
Usability	Role-focused dashboards, searchable tables, clear workflow queues, and status labels reduce staff effort.	High
Performance	Server-side filtering, pagination, database indexes, and aggregate dashboard queries keep pages responsive.	Medium
Extensibility	External integrations are isolated behind services, allowing future payment gateway, SSO, and mobile clients.	Medium

Attribute	Design Support	Priority
Auditability	Critical financial and occupancy actions write audit entries with actor, timestamp, target, and change details.	High

11 Requirement–Design Traceability

Req. ID	Requirement	Design Element	Verification Target
FR-01	Authentication and roles	Identity and Access module, Django auth, UserProfile, role groups, permission decorators.	Login and role-access tests.
FR-02	Student profile management	Resident Management module, Student model, student CRUD views.	CRUD and import tests.
FR-03	Room and bed inventory	Room Inventory module, Building, Floor, Room, and Bed models.	Room/bed CRUD tests.
FR-04	Automatic assignment	Assignment service, scoring strategy, AssignmentCandidate structure.	Scoring rule tests.
FR-05	Manual assignment override	Assignment service, admin views, audit log reason field.	Override workflow test.
FR-06	Check-in workflow	Check-In / Check-Out module, Assignment state change, Inspection model.	Check-in integration test.
FR-07	Check-out workflow	Check-In / Check-Out module, checkout settlement algorithm, penalty invoice lines.	Checkout settlement test.
FR-08	Billing and invoices	Billing module, Invoice and InvoiceLine models.	Invoice generation test.
FR-09	Payment evidence upload	Payment Review module, PaymentEvidence model, file storage gateway.	Upload and validation test.
FR-10	Payment review and confirmation	Payment review service, email digest, confirmation transaction, audit log.	Confirm/reject tests.
FR-11	Notifications and announcements	Notification module, Announcement and Notification models.	Notification delivery test.
FR-12	Dashboard and reports	Reporting module, dashboard queries, CSV report endpoints.	Aggregate report tests.
FR-13	Audit trail	AuditLog model and service-level audit writing.	Audit record assertions.
FR-14	Synthetic demo data	Demo Data module and seed command.	Seed command test.

Req. ID	Requirement	Design Element	Verification Target
NFR-01	Security	Authentication middleware, role checks, restricted file access, file validation.	Negative access tests.
NFR-02	Reliability	Database transactions, row locks, model constraints.	Concurrency test.
NFR-03	Usability	Role-focused navigation, placeholder wireframes, searchable lists.	Demo task walkthrough.
NFR-04	Performance	Pagination, indexes, aggregate queries.	Query-count and timing checks.
NFR-05	Maintainability	App modularization, service layer, design patterns.	Design review.
NFR-06	Extensibility	Integration gateways and strategy interfaces.	Extension review.
NFR-07	Auditability	AuditLog and payment/assignment decision tracking.	Audit trail tests.
NFR-08	Portability	Environment variables and Docker-ready deployment design.	Fresh setup run.

12 Validation / Testing Strategy

12.1 Design Review

The design should be reviewed against the original project objectives, MVP scope, and traceability matrix. Reviewers should verify that every major workflow has a responsible module, data model, user interface path, and test strategy. The review checklist should include:

- Each diagram uses the correct notation for its purpose: use cases inside a system boundary, UML activity and state symbols, DFD processes/data stores/external entities, and ERD cardinalities.
- No rendered diagram has overlapping text, crossed labels, or unreadable connector labels.
- Every entity relationship in the ERD has an explicit cardinality such as 1:1, 1:N, or N:M through an associative entity.
- Each lifecycle state transition is represented in both the state diagram and the service-level pseudocode.
- UI figures remain labelled as placeholders until replaced with real screenshots from the working demo.

12.2 Test Plan Overview

Test Area	Focus	Method
Unit tests	Service functions, status transitions, balance calculations, scoring rules.	Django test cases.
Integration tests	Assignment, check-in, billing, payment confirmation, check-out settlement.	Database-backed tests.
Permission tests	Role access for student, admin, finance, and system administrator.	View and API tests.
UI tests	Core workflows through browser pages.	Manual demo script or browser automation.
Data tests	Seed data consistency and report totals.	Management command tests.
Security tests	Unauthorized access, file type rejection, cross-user data access.	Negative tests.

12.3 Key Test Cases

1. Student can sign in and view only their own assignment and invoices.
2. Admin creates a room with four beds and sees all beds in available status.
3. Assignment recommendation excludes occupied, inactive, and gender-incompatible rooms.
4. Two admins cannot reserve the same bed at the same time.
5. Check-in changes assignment to active and bed to occupied.
6. Finance generates an invoice and student uploads payment evidence.
7. Payment upload creates pending payment, stores evidence, and queues review email.
8. Finance confirms payment and invoice status changes to paid when the confirmed amount is sufficient.
9. Rejected payment does not reduce outstanding invoice balance.
10. Check-out inspection creates penalties, settles deposit, completes assignment, and releases bed.
11. Dashboard counts match database records after seed data is loaded.
12. Unauthorized users cannot access staff pages or payment confirmation endpoints.

12.4 Acceptance Criteria

- The demo can be started from a clean database with synthetic data.
- Student, admin, and finance accounts can complete their main workflows.
- The system prevents double booking and unauthorized payment confirmation.
- Dashboard metrics update after assignment, payment, and check-out actions.
- Payment evidence review includes email notification and manual confirmation.
- The report and implementation remain aligned through the traceability matrix.

13 Conclusion

The proposed Student Dormitory Management System provides a practical MVP design for digitizing dormitory operations. The design centralizes student records, room and bed inventory, occupancy workflows, billing, payment review, announcements, notifications, reporting, and demo data generation. The expected benefit is a single source of operational truth: dorm staff can see current occupancy, finance staff can trace invoices and payment evidence, and students can view assignments and payment status without relying on separate spreadsheets or messages.

The report is ready to guide implementation because each major requirement is connected to an architecture layer, data model, interface path, service method, and verification target. The most important correctness risks, including bed double booking, unauthorized payment confirmation, invalid lifecycle transitions, and lost audit evidence, are handled through role permissions, service-layer transactions, database constraints, and audit logging.

For the project demo, the server-rendered Django stack is deliberately practical. It can run locally with Docker Compose, PostgreSQL, seeded demo data, local media storage, and a console or SMTP email backend. The same structure can later move to a low-cost VM or container host with managed PostgreSQL, S3-compatible media storage, TLS at a reverse proxy, and environment-based configuration. Future improvements can add a payment gateway, university SSO, online housing applications, roommate matching, visitor management, background job processing, mobile access, and a separate React or mobile client using the documented `/api/v1` endpoints.